

FINÁLNÍ PROJEKT č. 2

Testování REST API

Celkem testovacích případů: 96

Celkem nalezených bugů: 21

Autor: Kristína Belejčáková
Datum: 07.06.2026

Obsah

1. Zadání

2. Testovací scénáře — GET (12 případů)

2.1 Pozitivní testy (TC_GET_01 – TC_GET_03)

2.2 Negativní testy (TC_GET_04 – TC_GET_12)

3. Testovací scénáře — DELETE (12 případů)

3.1 Pozitivní testy (TC_DEL_01 – TC_DEL_03)

3.2 Negativní testy (TC_DEL_04 – TC_DEL_12)

4. Testovací scénáře — PUT (26 případů)

4.1 Pozitivní testy (TC_PUT_01 – TC_PUT_12)

4.2 Negativní testy (TC_PUT_13 – TC_PUT_26)

5. Testovací scénáře — POST (46 případů)

5.1 Pozitivní testy (TC_POST_01 – TC_POST_23)

5.2 Negativní testy (TC_POST_24 – TC_POST_32)

5.3 Hraniční testy (TC_POST_33 – TC_POST_46)

6. Exekuce testů

7. Bug report

Zadání

Popis projektu, cíl testování a přístupové údaje.

Cílem finálního projektu je otestovat funkčnost aplikace, která slouží k manipulaci s daty o studentech. Aplikace má rozhraní REST API, které umožňuje vytvoření, získání, aktualizaci a smazání dat.

Přístupové údaje:

Databáze — schema	students
Host	test-app.engeto.cz
Port	5433
REST API URL	https://test-app.engeto.cz/students
Dokumentace	https://test-app.engeto.cz/docs

Poznámky:

Data musí být správně uložena v databázi a z ní správně získávána. Každý testovací scénář obsahuje kroky, testovací data, DBeaver příkaz pro DB verifikaci, očekávaný výsledek a stav.

● Testovací scénáře — GET

Metoda GET slouží k získávání dat ze serveru. Celkem: 12 testovacích případů. Pro všechny testovací případy platí vstupní podmínky — zvolená metoda GET, do pole URL vložený odkaz s správně zadaným ID..

Pozitivní testy (TC_GET_01 – TC_GET_03)

GET TC_GET_01 — Získání existujícího studenta podle ID	
Kroky	1. Vstupní podmínky jsou zadány. 2. Nastavím metodu GET a URL v Postman. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	https://test-app.engeto.cz/students/133
DBeaver příkaz	<pre>SELECT * FROM students WHERE id = 133</pre>
Očekávaný výsledek	200 OK — response obsahuje správná data studenta odpovídající záznamu v DB (firstName, lastName, email, isEuCitizen, numberOfFailedStudies).
Skutečný výsledek	API vrátilo 200 OK
Stav	Passed

GET TC_GET_02 — Získání studenta — ověření všech polí v response	
Kroky	1. Vstupní podmínky jsou zadány. 2. Nastavím metodu GET a URL v Postman. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu. 5. Porovnám každé pole v response se záznamem v DB.
URL	https://test-app.engeto.cz/students/133
DBeaver příkaz	<pre>SELECT * FROM students WHERE id = 133</pre>
Očekávaný výsledek	200 OK — všechna pole v response (id, firstName, lastName, email, isEuCitizen, numberOfFailedStudies) přesně odpovídají hodnotám v DB.
Skutečný výsledek	API vrátilo 200 OK
Stav	Passed

GET TC_GET_03 — Získání studenta s ID obsahujícím mezery (trim)	
Kroky	1. Vstupní podmínky jsou zadány. 2. Nastavím metodu GET a URL v Postman. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	https://test-app.engeto.cz/students/133
DBeaver příkaz	<pre>SELECT * FROM students WHERE id = 133</pre>
Očekávaný výsledek	200 OK — API ořízne mezery kolem ID a vrátí správná data studenta.
Skutečný výsledek	API vrátilo 200 OK

Stav	Passed
------	--------

Negativní testy (TC_GET_04 – TC_GET_12)

GET TC_GET_04 — Získání neexistujícího studenta	
Kroky	1. Vstupní podmínky jsou zadány. 2. Nastavím metodu GET a URL v Postman. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	Nejvyšší ID je 171. Zadávám <code>https://test-app.engeto.cz/students/175</code>
DBeaver příkaz	<code>SELECT MAX(id) FROM students</code>
Očekávaný výsledek	404 Not Found — response obsahuje chybovou zprávu; code = NOT_FOUND.
Skutečný výsledek	API vrátilo 500 Internal Server Error
Stav	Failed

GET TC_GET_05 — ID = 0 (hraniční — neexistující)	
Kroky	1. Vstupní podmínky jsou zadány. 2. Nastavím metodu GET a URL v Postman. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	<code>https://test-app.engeto.cz/students/0</code>
DBeaver příkaz	<code>SELECT * FROM students WHERE id = 0</code>
Očekávaný výsledek	404 Not Found — student s id = 0 neexistuje; code = NOT_FOUND.
Skutečný výsledek	API vrátilo 500 Internal Server Error
Stav	Failed

GET TC_GET_06 — ID = záporné číslo (-1)	
Kroky	1. Vstupní podmínky jsou zadány. 2. Nastavím metodu GET a URL v Postman. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	<code>https://test-app.engeto.cz/students/-1</code>
DBeaver příkaz	<code>SELECT * FROM students WHERE id = -1</code>
Očekávaný výsledek	404 Not Found nebo 422 — záporné ID není platné; chybová zpráva.
Skutečný výsledek	API vrátilo 500 Internal Server Error. Záporné ID není korektně ošetřeno a způsobuje chybu serveru.
Stav	Failed

GET TC_GET_07 — ID jako text (xyz)

Kroky	1. Vstupní podmínky jsou zadány. 2. Nastavím metodu GET a URL v Postman. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	https://test-app.engeto.cz/students/xyz
DBeaver příkaz	<code>-- DB není volána; chyba nastane na úrovni validace vstupu.</code>
Očekávaný výsledek	422 Unprocessable Entity — textový řetězec není platné ID; code = VALIDATION_ERROR.
Skutečný výsledek	API vrátilo 422 Unprocessable Entity
Stav	Passed

GET TC_GET_08 — ID jako desetinné číslo (4.7)

Kroky	1. Vstupní podmínky jsou zadány. 2. Nastavím metodu GET a URL v Postman. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	https://test-app.engeto.cz/students/4.7
DBeaver příkaz	<code>-- DB není volána; chyba nastane na úrovni validace vstupu.</code>
Očekávaný výsledek	422 Unprocessable Entity — desetinné číslo není platné integer ID; code = VALIDATION_ERROR.
Skutečný výsledek	API vrátilo 422 Unprocessable Entity
Stav	Passed

GET TC_GET_09 — ID obsahuje speciální znaky (*>\$|)

Kroky	1. Vstupní podmínky jsou zadány. 2. Nastavím metodu GET a URL v Postman. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	https://test-app.engeto.cz/students/*>\$
DBeaver příkaz	<code>-- DB není volána; chyba nastane na úrovni validace vstupu.</code>
Očekávaný výsledek	422 Unprocessable Entity — speciální znaky nejsou platné ID; code = VALIDATION_ERROR.
Skutečný výsledek	API vrátilo 422 Unprocessable Entity
Stav	Passed

GET TC_GET_10 — Prázdná hodnota ID

Kroky	1. Vstupní podmínky jsou zadány. 2. Nastavím metodu GET a URL v Postman. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	https://test-app.engeto.cz/students/

DBeaver příkaz	<code>-- DB není volána.</code>
Očekávaný výsledek	422 Unprocessable Entity — prázdná hodnota ID není povolena; code = VALIDATION_ERROR.
Skutečný výsledek	API vrátilo 405 Method Not Allowed. Prázdné ID nebylo zpracováno dle očekávání 422.
Stav	Failed

GET TC_GET_11 — Velmi vysoké neexistující ID (999999999)

Kroky	1. Vstupní podmínky jsou zadány. 2. Nastavím metodu GET a URL v Postman. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	<code>https://test-app.engeto.cz/students/999999999</code>
DBeaver příkaz	<code>SELECT * FROM students WHERE id = 999999999</code>
Očekávaný výsledek	404 Not Found — student s tímto ID neexistuje; code = NOT_FOUND.
Skutečný výsledek	API vrátilo 500 Internal Server Error
Stav	Failed

GET TC_GET_12 — Ověření, že GET po DELETE vrátí 404

Kroky	1. Vytvoř studenta přes POST, ulož ID. 2. Odešli DELETE /students/{id}. 3. Odešli GET /students/{id} pro stejné ID. 4. Zkontroluj status kód.
URL	<code>https://test-app.engeto.cz/students/134</code>
DBeaver příkaz	<code>SELECT * FROM students WHERE id = 134</code>
Očekávaný výsledek	404 Not Found — po smazání student neexistuje; systém konzistentně reflektuje smazání.
Skutečný výsledek	API po vykonání DELETE (zpráva: Student deleted successfully) stále vrací při GET požadavku data studenta s kódem 200 OK.
Stav	Failed

● Testovací scénáře — DELETE

Metoda DELETE slouží ke smazání záznamů. Celkem: 12 testovacích případů. Pro všechny testovací případy platí vstupní podmínky — zvolená metoda DELETE, do pole pro URL zadáný odkaz s správným ID.

Pozitivní testy (TC_DEL_01 – TC_DEL_03)

DELETE TC_DEL_01 — Smazání existujícího studenta	
Kroky	1. Vytvoř studenta přes POST, ulož přidělené ID. 2. Nastavím metodu DELETE a URL v Postman. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	https://test-app.engeto.cz/students/134
DBeaver příkaz	<pre>SELECT * FROM students WHERE id = 134</pre>
Očekávaný výsledek	200 OK — response potvrzuje úspěšné smazání; záznam v DB neexistuje.
Skutečný výsledek	API vrátilo 200 OK, student byl odstraněn.
Stav	Passed

DELETE TC_DEL_02 — Ověření, že GET po DELETE vrátí 404 (regresní test)	
Kroky	1. Vytvoř studenta přes POST, ulož ID. 2. Odešli DELETE — ověř status 200. 3. Odešli GET na stejné ID. 4. Zkontroluj status kód GET requestu.
URL	https://test-app.engeto.cz/students/134
DBeaver příkaz	<pre>SELECT * FROM students WHERE id = 134</pre>
Očekávaný výsledek	DELETE: 200 OK. Následný GET: 404 Not Found. Systém konzistentně reflektuje smazání.
Skutečný výsledek	API po vykonání DELETE vrací kód 200 OK, ale při následném GET znovu hlásí 200 OK místo 404 Not Found — data studenta jsou zobrazena.
Stav	Failed

DELETE TC_DEL_03 — Smazání studenta s ID obsahujícím mezery	
Kroky	1. Vytvoř studenta přes POST, ulož ID. 2. Nastavím metodu DELETE a URL v Postman. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	https://test-app.engeto.cz/students/ 140
DBeaver příkaz	<pre>SELECT * FROM students WHERE id = 140</pre>
Očekávaný výsledek	200 OK — API ořízne mezery kolem ID a smaže správný záznam.
Skutečný výsledek	API vrátilo 200 OK

Stav	Passed
------	--------

Negativní testy (TC_DEL_04 – TC_DEL_12)

DELETE TC_DEL_04 — Smazání neexistujícího studenta (id = 1000)	
Kroky	1. Vstupní podmínky jsou zadány. 2. Nastavím metodu DELETE a URL v Postman. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	https://test-app.engeto.cz/students/1000
DBeaver příkaz	<code>SELECT * FROM students WHERE id = 1000</code>
Očekávaný výsledek	404 Not Found — student s id = 1000 neexistuje; chybová zpráva.
Skutečný výsledek	API vrátilo 500 Internal Server Error
Stav	Failed

DELETE TC_DEL_05 — Smazání studenta s id = 0	
Kroky	1. Vstupní podmínky jsou zadány. 2. Nastavím metodu DELETE a URL v Postman. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	https://test-app.engeto.cz/students/0
DBeaver příkaz	<code>SELECT * FROM students WHERE id = 0</code>
Očekávaný výsledek	404 Not Found — student s id = 0 neexistuje; chybová zpráva.
Skutečný výsledek	API vrátilo 500 Internal Server Error
Stav	Failed

DELETE TC_DEL_06 — Smazání studenta se záporným id (-53)	
Kroky	1. Vstupní podmínky jsou zadány. 2. Nastavím metodu DELETE a URL v Postman. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	https://test-app.engeto.cz/students/-53
DBeaver příkaz	<code>-- DB není volána při validační chybě.</code>
Očekávaný výsledek	422 Unprocessable Entity — záporné ID není platné; code = VALIDATION_ERROR.
Skutečný výsledek	API vrátilo 500 Internal Server Error
Stav	Failed

DELETE TC_DEL_07 — Smazání studenta s textovým id (abc)

Kroky	1. Vstupní podmínky jsou zadány. 2. Nastavím metodu DELETE a URL v Postman. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	https://test-app.engeto.cz/students/abc
DBeaver příkaz	<code>-- DB není volána při validační chybě.</code>
Očekávaný výsledek	422 Unprocessable Entity — textový řetězec není platné ID; code = VALIDATION_ERROR.
Skutečný výsledek	API vrátilo 422 Unprocessable Entity
Stav	Passed

DELETE TC_DEL_08 — Smazání studenta s desetinným id (53.5)

Kroky	1. Vstupní podmínky jsou zadány. 2. Nastavím metodu DELETE a URL v Postman. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	https://test-app.engeto.cz/students/53.5
DBeaver příkaz	<code>-- DB není volána při validační chybě.</code>
Očekávaný výsledek	422 Unprocessable Entity — desetinné číslo není platné integer ID; code = VALIDATION_ERROR.
Skutečný výsledek	API vrátilo 422 Unprocessable Entity
Stav	Passed

DELETE TC_DEL_09 — Smazání studenta se speciálními znaky v id (*>\$|)

Kroky	1. Vstupní podmínky jsou zadány. 2. Nastavím metodu DELETE a URL v Postman. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	https://test-app.engeto.cz/students/*>\$
DBeaver příkaz	<code>-- DB není volána při validační chybě.</code>
Očekávaný výsledek	422 Unprocessable Entity — speciální znaky nejsou platné ID; code = VALIDATION_ERROR.
Skutečný výsledek	API vrátilo 422 Unprocessable Entity
Stav	Passed

DELETE TC_DEL_10 — Opakované smazání stejného studenta (dvojitě DELETE)

Kroky	1. Vytvoř studenta přes POST, ulož ID. 2. Odešli první DELETE — ověř status 200. 3. Odešli druhý DELETE na stejné ID. 4. Zkontroluj status kód druhého requestu.
URL	https://test-app.engeto.cz/students/136

DBeaver příkaz	<code>SELECT * FROM students WHERE id = 136</code>
Očekávaný výsledek	První DELETE: 200 OK. Druhý DELETE: 404 Not Found — student již neexistuje.
Skutečný výsledek	API správně odpovídá při DELETE — 200 OK. Při opakovaném GET (po DELETE) stále studenta zobrazí s kódem 200 OK.
Stav	Failed

DELETE TC_DEL_11 — Smazání studenta — ověření, že ostatní záznamy nejsou ovlivněny

Kroky	1. Vytvoř 2 studenty přes POST, ulož jejich ID. 2. Odešli DELETE pro studenta A. 3. Odešli GET pro studenta B. 4. Zkontroluj, že student B stále existuje.
URL	Student A: https://test-app.engeto.cz/students/173 Student B: https://test-app.engeto.cz/students/174
BODY	Student A: { "name": "Anna Nováková", "email": "anna.novakova@student.sk" } Student B: { "name": "Peter Horváth", "email": "peter.horvath@student.sk" }
DBeaver příkaz	<code>A: SELECT * FROM students WHERE email = 'anna.novakova@student.sk'</code> <code>B: SELECT * FROM students WHERE email = 'peter.horvath@student.sk'</code>
Očekávaný výsledek	200 OK pro DELETE studenta A. GET studenta B vrátí 200 OK s nezměněnými daty.
Skutečný výsledek	Po smazání studenta A (ID 173) došlo k poškození dat studenta B (ID 174). API vrátilo hodnoty null u jména a příjmení.
Stav	Failed

DELETE TC_DEL_12 — Smazání studenta — velmi vysoké neexistující ID

Kroky	1. Vstupní podmínky jsou zadány. 2. Nastavím metodu DELETE a URL v Postman. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	https://test-app.engeto.cz/students/101010101010101010
DBeaver příkaz	<code>SELECT * FROM students WHERE id = 101010101010101010</code>
Očekávaný výsledek	404 Not Found — student s tímto ID neexistuje; chybová zpráva.
Skutečný výsledek	API při pokusu o smazání studenta s neplatným ID vrátilo 500 Internal Server Error namísto očekávaného kódu 404 Not Found.
Stav	Failed

● Testovací scénáře — PUT

Metoda PUT slouží k aktualizaci existujících záznamů. Celkem: 26 testovacích případů. Pro všechny testovací případy platí vstupní podmínky — zvolená metoda PUT a s správným formátem v sekci BODY. Vložený URL odkaz se správně přiřazeným ID.

Pozitivní testy (TC_PUT_01 – TC_PUT_12)

PUT TC_PUT_01 — Aktualizace studenta — všechna platná pole	
Kroky	1. Vytvoř studenta přes POST, ulož ID. 2. Pole BODY obsahuje údaje pro tento testovací případ. Změna je v emailu: "@test1" na "@test2". 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	https://test-app.engeto.cz/students/172
Body	{ "firstName": "Petr", "lastName": "Novák", "email": "petr.novak@test2.cz", "isEuCitizen": true, "numberOfFailedStudies": 2, "zipCode": "10000" }
DBeaver příkaz	<pre>SELECT * FROM students WHERE id = 172</pre>
Očekávaný výsledek	200 OK — response obsahuje aktualizované hodnoty všech polí; změny jsou uloženy v DB.
Skutečný výsledek	API vrátilo 200 OK
Stav	Passed

PUT TC_PUT_02 — Částečná aktualizace — pouze firstName	
Kroky	1. Vytvoř studenta přes POST, zaznamenej původní hodnoty. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	https://test-app.engeto.cz/students/172
Body	{ "firstName": "Pavel" }
DBeaver příkaz	<pre>SELECT * FROM students WHERE id = 172</pre>
Očekávaný výsledek	200 OK — firstName aktualizováno; všechna ostatní pole zůstávají beze změny.
Skutečný výsledek	API vrátilo 200 OK
Stav	Passed

PUT TC_PUT_03 — Částečná aktualizace — pouze lastName	
Kroky	1. Vytvoř studenta přes POST. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	https://test-app.engeto.cz/students/172
Body	{ "lastName": "Svoboda" }

DBeaver příkaz	<code>SELECT * FROM students WHERE id = 172</code>
Očekávaný výsledek	200 OK — lastName aktualizováno; ostatní pole beze změny.
Skutečný výsledek	API vrátilo 200 OK
Stav	Passed

PUT TC_PUT_04 — Částečná aktualizace — pouze email	
Kroky	1. Vytvoř studenta přes POST. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	https://test-app.engeto.cz/students/172
Body	{ "email": "nova.adresa@test.cz" }
DBeaver příkaz	<code>SELECT * FROM students WHERE id = 172</code>
Očekávaný výsledek	200 OK — email aktualizován; ostatní pole beze změny.
Skutečný výsledek	API vrátilo 200 OK
Stav	Passed

PUT TC_PUT_05 — Částečná aktualizace — pouze isEuCitizen	
Kroky	1. Vytvoř studenta přes POST s isEuCitizen = true. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	https://test-app.engeto.cz/students/172
Body	{ "isEuCitizen": false }
DBeaver příkaz	<code>SELECT * FROM students WHERE id = 172</code>
Očekávaný výsledek	200 OK — isEuCitizen aktualizováno na false; ostatní pole beze změny.
Skutečný výsledek	API vrátilo 200 OK
Stav	Passed

PUT TC_PUT_06 — Částečná aktualizace — pouze numberOfFailedStudies	
Kroky	1. Vytvoř studenta přes POST. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	https://test-app.engeto.cz/students/172
Body	{ "numberOfFailedStudies": 5 }
DBeaver příkaz	<code>SELECT * FROM students WHERE id = 172</code>

Očekávaný výsledek	200 OK — numberOfFailedStudies aktualizováno na 5; ostatní pole beze změny.
Skutečný výsledek	API vrátilo 200 OK
Stav	Passed

PUT TC_PUT_07 — Částečná aktualizace — pouze zipCode

Kroky	1. Vytvoř studenta přes POST. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	https://test-app.engeto.cz/students/172
Body	{ "zipCode": "60200" }
DBeaver příkaz	<code>SELECT * FROM students WHERE id = 172</code>
Očekávaný výsledek	200 OK — zipCode aktualizováno; ostatní pole beze změny.
Skutečný výsledek	API vrátilo 200 OK, ale zipCode se nepřepsalo ani v DB.
Stav	Passed

PUT TC_PUT_08 — Aktualizace firstName = null

Kroky	1. Vytvoř studenta přes POST. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	https://test-app.engeto.cz/students/172
Body	{ "firstName": null }
DBeaver příkaz	<code>SELECT * FROM students WHERE id = 172</code>
Očekávaný výsledek	200 OK — firstName aktualizováno na null; hodnota null zachována v DB.
Skutečný výsledek	API vrátilo 422 Unprocessable Entity, chybová zpráva uvádí, že musí být poskytnuto nejméně jedno pole pro aktualizaci.
Stav	Failed

PUT TC_PUT_09 — Aktualizace email = null

Kroky	1. Vytvoř studenta přes POST. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	https://test-app.engeto.cz/students/172
Body	{ "email": null }
DBeaver příkaz	<code>SELECT * FROM students WHERE id = 172</code>
Očekávaný výsledek	200 OK — email aktualizován na null; hodnota null zachována v DB.

Skutečný výsledek	API vrátilo 422 Unprocessable Entity, chybová zpráva uvádí, že musí být poskytnuto nejméně jedno pole pro aktualizaci.
Stav	Failed

PUT TC_PUT_10 — Aktualizace isEuCitizen = null

Kroky	1. Vytvoř studenta přes POST. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	https://test-app.engeto.cz/students/172
Body	{ "isEuCitizen": null }
DBeaver příkaz	<pre>SELECT * FROM students WHERE id = 172</pre>
Očekávaný výsledek	200 OK — isEuCitizen aktualizováno na null; hodnota null zachována v DB (nesmí být konvertována na false).
Skutečný výsledek	API vrátilo 422 Unprocessable Entity, chybová zpráva uvádí, že musí být poskytnuto nejméně jedno pole pro aktualizaci.
Stav	Failed

PUT TC_PUT_11 — Aktualizace numberOfFailedStudies = null

Kroky	1. Vytvoř studenta přes POST. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	https://test-app.engeto.cz/students/172
Body	{ "numberOfFailedStudies": null }
DBeaver příkaz	<pre>SELECT * FROM students WHERE id = 172</pre>
Očekávaný výsledek	200 OK — numberOfFailedStudies aktualizováno na null; hodnota null zachována v DB (nesmí být konvertována na 0).
Skutečný výsledek	API vrátilo 422 Unprocessable Entity, chybová zpráva uvádí, že musí být poskytnuto nejméně jedno pole pro aktualizaci.
Stav	Failed

PUT TC_PUT_12 — Aktualizace lastName = prázdný string

Kroky	1. Vytvoř studenta přes POST. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	https://test-app.engeto.cz/students/172
Body	{ "lastName": "" }
DBeaver příkaz	<pre>SELECT * FROM students WHERE id = 172</pre>
Očekávaný výsledek	200 OK — lastName aktualizováno na prázdný string nebo null dle implementace. Ostatní pole beze změny.
Skutečný výsledek	API vrátilo 200 OK

Stav	Passed
------	--------

Negativní testy (TC_PUT_13 – TC_PUT_26)

PUT TC_PUT_13 — Aktualizace neexistujícího studenta (id = 99999)	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	https://test-app.engeto.cz/students/99999
Body	{ "firstName": "Karel" }
DBeaver příkaz	<code>SELECT * FROM students WHERE id = 99999</code>
Očekávaný výsledek	404 Not Found — student s id = 99999 neexistuje; chybová zpráva.
Skutečný výsledek	API vrátilo 404 Not Found
Stav	Passed

PUT TC_PUT_14 — Aktualizace studenta s id = 0	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	https://test-app.engeto.cz/students/0
Body	{ "firstName": "Karel" }
DBeaver příkaz	<code>SELECT * FROM students WHERE id = 0</code>
Očekávaný výsledek	404 Not Found — student s id = 0 neexistuje; chybová zpráva.
Skutečný výsledek	API vrátilo 404 Not Found
Stav	Passed

PUT TC_PUT_15 — Aktualizace studenta se záporným id (-1)	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	https://test-app.engeto.cz/students/-1
Body	{ "firstName": "Karel" }
DBeaver příkaz	<code>-- DB není volána při validační chybě.</code>
Očekávaný výsledek	422 Unprocessable Entity — záporné ID není platné; code = VALIDATION_ERROR.

Skutečný výsledek	API vrátilo 404 Not Found. Záporné ID nebylo zpracováno jako validační chyba (422), ale jako chyba nenalezeného studenta.
Stav	Failed

PUT TC_PUT_16 — Aktualizace studenta s textovým id (abc)	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	https://test-app.engeto.cz/students/abc
Body	{ "firstName": "Karel" }
DBeaver příkaz	-- DB není volána při validační chybě.
Očekávaný výsledek	422 Unprocessable Entity — textové ID není platné; code = VALIDATION_ERROR.
Skutečný výsledek	API vrátilo 422 Unprocessable Entity
Stav	Passed

PUT TC_PUT_17 — Aktualizace studenta s desetinným id (50.5)	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	https://test-app.engeto.cz/students/50.5
Body	{ "firstName": "Karel" }
DBeaver příkaz	-- DB není volána při validační chybě.
Očekávaný výsledek	422 Unprocessable Entity — desetinné ID není platné; code = VALIDATION_ERROR.
Skutečný výsledek	API vrátilo 422 Unprocessable Entity
Stav	Passed

PUT TC_PUT_18 — Aktualizace s prázdným tělem requestu	
Kroky	1. Vytvoř studenta přes POST, ulož ID. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	https://test-app.engeto.cz/students/133
Body	{}
DBeaver příkaz	SELECT * FROM students WHERE id = [ID vytvořeného studenta]
Očekávaný výsledek	422 Unprocessable Entity — prázdné tělo requestu; je vyžadováno alespoň jedno pole; code = VALIDATION_ERROR. Záznam v DB beze změny.
Skutečný výsledek	API vrátilo 422 Unprocessable Entity

Stav	Passed
------	--------

PUT TC_PUT_19 — Aktualizace s neplatným JSON	
Kroky	1. Vytvoř studenta přes POST, ulož ID. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	https://test-app.engeto.cz/students/172
Body	"firstName": "Pepa", "lastName": "Novák"
DBeaver příkaz	<code>SELECT * FROM students WHERE id = [ID vytvořeného studenta]</code>
Očekávaný výsledek	422 Unprocessable Entity — poškozený JSON; code = VALIDATION_ERROR. Záznam v DB beze změny.
Skutečný výsledek	API vrátilo 422 Unprocessable Entity
Stav	Passed

PUT TC_PUT_20 — Aktualizace s neočekávaným polem	
Kroky	1. Vytvoř studenta přes POST, ulož ID. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	https://test-app.engeto.cz/students/172
Body	{ "unknownField": "testovací hodnota" }
DBeaver příkaz	<code>SELECT * FROM students WHERE id = [ID vytvořeného studenta]</code>
Očekávaný výsledek	422 Unprocessable Entity — neočekávané pole; code = VALIDATION_ERROR. Záznam v DB beze změny.
Skutečný výsledek	API vrátilo 422 Unprocessable Entity
Stav	Passed

PUT TC_PUT_21 — Aktualizace isEuCitizen = "true" (string místo boolean)	
Kroky	1. Vytvoř studenta přes POST, ulož ID. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	https://test-app.engeto.cz/students/172
Body	{ "isEuCitizen": "true" }
DBeaver příkaz	<code>SELECT * FROM students WHERE id = 172</code>
Očekávaný výsledek	422 Unprocessable Entity — isEuCitizen musí být boolean; code = VALIDATION_ERROR. Záznam v DB beze změny.
Skutečný výsledek	API vrátilo 200 OK, ale hodnotu na "true" nezměnilo. Místo validační chyby 422 API řetězec true ignorovalo nebo chybně interpretovalo jako false.
Stav	Failed

PUT TC_PUT_22 — Aktualizace numberOfFailedStudies = záporná hodnota (-1)	
Kroky	1. Vytvoř studenta přes POST, ulož ID. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	https://test-app.engeto.cz/students/172
Body	{ "numberOfFailedStudies": -1 }
DBeaver příkaz	<code>SELECT * FROM students WHERE id = 172</code>
Očekávaný výsledek	422 Unprocessable Entity — záporná hodnota není platná; code = VALIDATION_ERROR. Záznam v DB beze změny.
Skutečný výsledek	API vrátilo 422 Unprocessable Entity
Stav	Passed

PUT TC_PUT_23 — Aktualizace numberOfFailedStudies = desetinné číslo (5.5)	
Kroky	1. Vytvoř studenta přes POST, ulož ID. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	https://test-app.engeto.cz/students/172
Body	{ "numberOfFailedStudies": 5.5 }
DBeaver příkaz	<code>SELECT * FROM students WHERE id = 172</code>
Očekávaný výsledek	422 Unprocessable Entity — desetinná hodnota není platná pro integer pole; code = VALIDATION_ERROR.
Skutečný výsledek	API vrátilo 422 Unprocessable Entity
Stav	Passed

PUT TC_PUT_24 — Aktualizace numberOfFailedStudies = string ("abc")	
Kroky	1. Vytvoř studenta přes POST, ulož ID. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
URL	https://test-app.engeto.cz/students/172
Body	{ "numberOfFailedStudies": "abc" }
DBeaver příkaz	<code>SELECT * FROM students WHERE id = 172</code>
Očekávaný výsledek	422 Unprocessable Entity — string není platný pro integer pole; code = VALIDATION_ERROR.
Skutečný výsledek	API vrátilo 422 Unprocessable Entity
Stav	Passed

PUT TC_PUT_25 — Aktualizace emailu na již existující email jiného studenta	
--	--

Kroky	1. Vytvoř 2 studenty přes POST (student A — ID 172 a B — ID 133), ulož jejich emaily. 2. Odešli PUT pro studenta A s emailem studenta B. 3. Zkontroluj status kód.
URL	https://test-app.engeto.cz/students/172
Body	{ "email": "kristina.b@seznam.cz" }
DBeaver příkaz	<pre>SELECT COUNT(*) FROM students WHERE email = 'kristina.b@seznam.cz'</pre>
Očekávaný výsledek	400 Bad Request — email již existuje; duplicitní email není povolen. V DB zůstává počet záznamů s tímto emailem = 1.
Skutečný výsledek	API vrátilo 200 OK. Systém povolil duplicitní email pro dva různé studenty (ID 133 a 172). Databáze nemá nastavenou unikátnost na sloupci email.
Stav	Failed

PUT TC_PUT_26 — Ověření, že partial PUT nemění nevyžádaná pole	
Kroky	1. Vytvoř studenta přes POST, zaznamenej všechny původní hodnoty. 2. Odešli PUT pouze s jedním polem (např. firstName). 3. Odešli GET a porovnej všechna pole s původními hodnotami.
URL	https://test-app.engeto.cz/students/172
Body	{ "firstName": "Nové jméno" }
DBeaver příkaz	<pre>SELECT * FROM students WHERE id = 172</pre>
Očekávaný výsledek	200 OK — pouze firstName je změněno; isEuCitizen, numberOfFailedStudies a všechna ostatní pole zůstávají identická jako před PUT requestem.
Skutečný výsledek	API vrátilo 200 OK. API změnilo jméno a ostatní pole zůstala nezměněna.
Stav	Passed

Testovací scénáře — POST

Metoda POST slouží k vytváření nových záznamů. Celkem: 46 testovacích případů. Pro všechny testovací případy platí vstupní podmínky — zvolená metoda POST, do pole BODY vložený v RAW ve formátu JSON.

Pozitivní testy (TC_POST_01 – TC_POST_23)

POST TC_POST_01 — Vytvoření nového studenta s validními údaji včetně zipCode	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{ "firstName": "Kristína", "lastName": "Belejčáková", "email": "kristina.b@seznam.cz", "isEuCitizen": true, "numberOfFailedStudies": 0, "zipCode": "43019" }
DBeaver příkaz	<code>SELECT * FROM students WHERE email = 'kristina.b@seznam.cz'</code>
Očekávaný výsledek	201 Created — response obsahuje přidělené ID a všechna zadaná data včetně zipCode = "43019". Pole zipCode musí být správně uloženo v DB.
Skutečný výsledek	API vrátilo 200 OK. API ignoruje vstupní parametr zipCode.
Stav	✗ Failed

POST TC_POST_02 — Vytvoření studenta bez firstName	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{ "lastName": "Greenová", "email": "rachel.green@seznam.cz", "isEuCitizen": true, "numberOfFailedStudies": 0 }
DBeaver příkaz	<code>SELECT * FROM students WHERE email = 'rachel.green@seznam.cz'</code>
Očekávaný výsledek	201 Created — dle dokumentace není žádné pole povinné. Student je vytvořen s firstName = null.
Skutečný výsledek	API vrátilo 201 Created s firstName = null.
Stav	✓ Passed

POST TC_POST_03 — Vytvoření studenta bez lastName	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{ "firstName": "Chandler", "email": "chandlerbing@seznam.cz", "isEuCitizen": true, "numberOfFailedStudies": 0 }
DBeaver příkaz	<code>SELECT * FROM students WHERE email = 'chandlerbing@seznam.cz'</code>
Očekávaný výsledek	201 Created — dle dokumentace není žádné pole povinné. Student je vytvořen s lastName = null.

Skutečný výsledek	API vrátilo 201 Created s lastName = null.
Stav	✓ Passed

POST TC_POST_04 — Vytvoření studenta bez e-mailu

Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{ "firstName": "Monica", "lastName": "Gellerová", "isEuCitizen": true, "numberOfFailedStudies": 0 }
DBeaver příkaz	<pre>SELECT * FROM students WHERE first_name = 'Monica' AND last_name = 'Gellerová'</pre>
Očekávaný výsledek	201 Created — dle dokumentace není žádné pole povinné. Student je vytvořen s email = null.
Skutečný výsledek	API vrátilo 201 Created s email = null.
Stav	✓ Passed

POST TC_POST_05 — Vytvoření studenta bez isEuCitizen

Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{ "firstName": "Ross", "lastName": "Geller", "email": "r.geller@seznam.cz", "numberOfFailedStudies": 0 }
DBeaver příkaz	<pre>SELECT * FROM students WHERE email = 'r.geller@seznam.cz'</pre>
Očekávaný výsledek	201 Created — dle dokumentace není žádné pole povinné. isEuCitizen má výchozí hodnotu dle implementace API (false).
Skutečný výsledek	API vrátilo 201 Created s isEuCitizen = false.
Stav	✓ Passed

POST TC_POST_06 — Vytvoření studenta bez numberOfFailedStudies

Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{ "firstName": "Joey", "lastName": "Tribbiani", "email": "joey.tribbiani@seznam.cz", "isEuCitizen": true }
DBeaver příkaz	<pre>SELECT * FROM students WHERE email = 'joey.tribbiani@seznam.cz'</pre>
Očekávaný výsledek	201 Created — numberOfFailedStudies má výchozí hodnotu dle implementace API (0).
Skutečný výsledek	API vrátilo 201 Created s numberOfFailedStudies = 0.
Stav	✓ Passed

POST TC_POST_07 — Vytvoření studenta s prázdným stringem firstName	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{ "firstName": "", "lastName": "Buffatová", "email": "pheobe.buffat@seznam.cz", "isEuCitizen": true, "numberOfFailedStudies": 0 }
DBeaver příkaz	<code>SELECT * FROM students WHERE email = 'pheobe.buffat@seznam.cz'</code>
Očekávaný výsledek	201 Created — prázdný string je validní hodnota dle dokumentace. firstName uloženo jako "" nebo null.
Skutečný výsledek	API vrátilo 201 Created. Student s číslem 140 je v DB dohledatelný podle emailu.
Stav	✓ Passed

POST TC_POST_08 — Vytvoření studenta s prázdným stringem lastName	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{ "firstName": "Mike", "lastName": "", "email": "mike.hannigan@seznam.cz", "isEuCitizen": true, "numberOfFailedStudies": 0 }
DBeaver příkaz	<code>SELECT * FROM students WHERE email = 'mike.hannigan@seznam.cz'</code>
Očekávaný výsledek	201 Created — prázdný string je validní. lastName uloženo jako null.
Skutečný výsledek	API vrátilo 201 Created. V DB je student dohledatelný podle ID 141 nebo emailem.
Stav	✓ Passed

POST TC_POST_09 — Vytvoření studenta s prázdným stringem e-mail	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{ "firstName": "Ben", "lastName": "Benjamin", "email": "", "isEuCitizen": true, "numberOfFailedStudies": 0 }
DBeaver příkaz	<code>SELECT * FROM students WHERE first_name = 'Ben' AND last_name = 'Benjamin'</code>
Očekávaný výsledek	201 Created — email uloženo jako "" nebo null.
Skutečný výsledek	API vrátilo 201 Created, email uložen jako "". V DB je student dohledatelný podle ID 142.
Stav	✓ Passed

POST TC_POST_10 — Vytvoření studenta s číslem místo firstName	
Kroky	1. Vstupní podmínky jsou zadány.

	2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{ "firstName": "123456789", "lastName": "Gunther", "email": "gunther@seznam.cz", "isEuCitizen": true, "numberOfFailedStudies": 0 }
DBeaver příkaz	<code>SELECT * FROM students WHERE email = 'gunther@seznam.cz'</code>
Očekávaný výsledek	201 Created — firstName je string; číselný string je platná hodnota.
Skutečný výsledek	API vrátilo 201 Created.
Stav	✓ Passed

POST TC_POST_11 — Vytvoření studenta s číslem místo lastName	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{ "firstName": "Janice", "lastName": "101010101010", "email": "janice.lit@seznam.cz", "isEuCitizen": true, "numberOfFailedStudies": 0 }
DBeaver příkaz	<code>SELECT * FROM students WHERE email = 'janice.lit@seznam.cz'</code>
Očekávaný výsledek	201 Created — lastName je string; číselný string je platná hodnota.
Skutečný výsledek	API vrátilo 201 Created.
Stav	✓ Passed

POST TC_POST_12 — Vytvoření studenta s neplatným formátem e-mailu (dvojitě @@)	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{ "firstName": "Jon", "lastName": "Snow", "email": "101010101010@@seznam.101010", "isEuCitizen": true, "numberOfFailedStudies": 0 }
DBeaver příkaz	<code>SELECT * FROM students WHERE first_name = 'Jon' AND last_name = 'Snow'</code>
Očekávaný výsledek	201 Created — email je string bez validace formátu. API záznam uloží.
Skutečný výsledek	API vrátilo 201 Created.
Stav	✓ Passed

POST TC_POST_13 — Vytvoření studenta se speciálními znaky místo firstName	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.

Body	{ "firstName": ">>!!!<<", "lastName": "Stark", "email": "arya.stark@seznam.cz", "isEuCitizen": true, "numberOfFailedStudies": 0 }
DBeaver příkaz	<code>SELECT * FROM students WHERE email = 'arya.stark@seznam.cz'</code>
Očekávaný výsledek	201 Created — firstName je string; speciální znaky jsou platná hodnota.
Skutečný výsledek	API vrátilo 201 Created.
Stav	✓ Passed

POST TC_POST_14 — Vytvoření studenta se speciálními znaky místo lastName	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{ "firstName": "Katniss", "lastName": ">>!!!<<", "email": "k.everdeen@seznam.cz", "isEuCitizen": true, "numberOfFailedStudies": 0 }
DBeaver příkaz	<code>SELECT * FROM students WHERE email = 'k.everdeen@seznam.cz'</code>
Očekávaný výsledek	201 Created — lastName je string; speciální znaky jsou platná hodnota.
Skutečný výsledek	API vrátilo 201 Created.
Stav	✓ Passed

POST TC_POST_15 — Vytvoření studenta se speciálními znaky místo e-mailu	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{ "firstName": "Peeta", "lastName": "Mellark", "email": ">\$β×÷", "isEuCitizen": true, "numberOfFailedStudies": 0 }
DBeaver příkaz	<code>SELECT * FROM students WHERE first_name = 'Peeta' AND last_name = 'Mellark'</code>
Očekávaný výsledek	201 Created — email je string bez validace formátu.
Skutečný výsledek	API vrátilo 201 Created.
Stav	✓ Passed

POST TC_POST_16 — Vytvoření studenta s neobvyklým (alfanumerickým) firstName	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{ "firstName": "A0N0N0A0C0R0E0S0T0", "lastName": "Crest", "email": "anna.crest@seznam.cz", "isEuCitizen": true, "numberOfFailedStudies": 0 }
DBeaver příkaz	<code>SELECT * FROM students WHERE email = 'anna.crest@seznam.cz'</code>

Očekávaný výsledek	201 Created — firstName je string bez omezení formátu ani délky. API záznam přijme.
Skutečný výsledek	API vrátilo 201 Created.
Stav	✓ Passed

POST TC_POST_17 — Vytvoření studenta s neobvyklým (alfanumerickým) lastName	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{ "firstName": "Hermiona", "lastName": "G0r0a0n0g0e0r0o0v0á0", "email": "h.grangerova@seznam.cz", "isEuCitizen": true, "numberOfFailedStudies": 0 }
DBeaver příkaz	<code>SELECT * FROM students WHERE email = 'h.grangerova@seznam.cz'</code>
Očekávaný výsledek	201 Created — lastName je string bez omezení. API záznam přijme.
Skutečný výsledek	API vrátilo 201 Created.
Stav	✓ Passed

POST TC_POST_18 — Vytvoření studenta s neplatným e-mailem (speciální znaky + neplatná doména)	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{ "firstName": "Lord", "lastName": "Voldemort", "email": "l0o0r0d0w0ldem0rt\$\$βα>*@seznam.ABC", "isEuCitizen": true, "numberOfFailedStudies": 0 }
DBeaver příkaz	<code>SELECT * FROM students WHERE first_name = 'Lord' AND last_name = 'Voldemort'</code>
Očekávaný výsledek	201 Created — email je string bez validace.
Skutečný výsledek	API vrátilo 201 Created.
Stav	✓ Passed

POST TC_POST_19 — Vytvoření studenta s isEuCitizen = false	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{ "firstName": "Harry", "lastName": "Potter", "email": "harry.p@seznam.cz", "isEuCitizen": false, "numberOfFailedStudies": 0 }
DBeaver příkaz	<code>SELECT * FROM students WHERE email = 'harry.p@seznam.cz'</code>
Očekávaný výsledek	201 Created — isEuCitizen = false je platná hodnota. V response i DB je false.

Skutečný výsledek	API vrátilo 201 Created.
Stav	✓ Passed

POST TC_POST_20 — Vytvoření studenta s isEuCitizen = true (platný boolean)	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{ "firstName": "Ron", "lastName": "Weasley", "email": "ron.w@seznam.cz", "isEuCitizen": true, "numberOfFailedStudies": 0 }
DBeaver příkaz	<code>SELECT * FROM students WHERE email = 'ron.w@seznam.cz'</code>
Očekávaný výsledek	201 Created — isEuCitizen = true je platná hodnota. V response i DB je true.
Skutečný výsledek	API vrátilo 201 Created.
Stav	✓ Passed

POST TC_POST_21 — Vytvoření studenta s numberOfFailedStudies = 100	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{ "firstName": "Draco", "lastName": "Malfoy", "email": "draco.m@seznam.cz", "isEuCitizen": true, "numberOfFailedStudies": 100 }
DBeaver příkaz	<code>SELECT * FROM students WHERE email = 'draco.m@seznam.cz'</code>
Očekávaný výsledek	201 Created — numberOfFailedStudies = 100 je platná hodnota.
Skutečný výsledek	API vrátilo 201 Created.
Stav	✓ Passed

POST TC_POST_22 — Vytvoření studenta s numberOfFailedStudies = 5	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{ "firstName": "Severus", "lastName": "Snape", "email": "severus.s@seznam.cz", "isEuCitizen": true, "numberOfFailedStudies": 5 }
DBeaver příkaz	<code>SELECT * FROM students WHERE email = 'severus.s@seznam.cz'</code>
Očekávaný výsledek	201 Created — numberOfFailedStudies = 5 je platná hodnota.
Skutečný výsledek	API vrátilo 201 Created.
Stav	✓ Passed

POST TC_POST_23 — Vytvoření studenta s numberOfFailedStudies = 0	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{ "firstName": "Sirius", "lastName": "Black", "email": "s.black@seznam.cz", "isEuCitizen": true, "numberOfFailedStudies": 0 }
DBeaver příkaz	<code>SELECT * FROM students WHERE email = 's.black@seznam.cz'</code>
Očekávaný výsledek	201 Created — numberOfFailedStudies = 0 je platná hodnota.
Skutečný výsledek	API vrátilo 201 Created.
Stav	✓ Passed

Negativní testy (TC_POST_24 – TC_POST_32)

POST TC_POST_24 — Vytvoření studenta s numberOfFailedStudies = -5 (záporný int)	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{ "firstName": "Luna", "lastName": "Lovegood", "email": "luna.l@seznam.cz", "isEuCitizen": true, "numberOfFailedStudies": -5 }
DBeaver příkaz	<code>SELECT * FROM students WHERE email = 'luna.l@seznam.cz'</code>
Očekávaný výsledek	422 Unprocessable Entity — záporná hodnota není platná; code = VALIDATION_ERROR. Záznam v DB nevznikne.
Skutečný výsledek	API vrátilo 422 Unprocessable Entity a záznam v DB nevznikl.
Stav	✓ Passed

POST TC_POST_25 — Vytvoření studenta s numberOfFailedStudies = 5.5 (float)	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{ "firstName": "Ginny", "lastName": "Weasley", "email": "ginny.w@seznam.cz", "isEuCitizen": true, "numberOfFailedStudies": 5.5 }
DBeaver příkaz	<code>SELECT * FROM students WHERE email = 'ginny.w@seznam.cz'</code>
Očekávaný výsledek	422 Unprocessable Entity — desetinná hodnota není platná pro integer pole; code = VALIDATION_ERROR.
Skutečný výsledek	API vrátilo 422 Unprocessable Entity a záznam v DB nevznikl.
Stav	✓ Passed

POST TC_POST_26 — Vytvoření studenta s numberOfFailedStudies = true (boolean)	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{ "firstName": "Neville", "lastName": "Longbottom", "email": "neville.l@seznam.cz", "isEuCitizen": true, "numberOfFailedStudies": true }
DBeaver příkaz	<code>SELECT * FROM students WHERE email = 'neville.l@seznam.cz'</code>
Očekávaný výsledek	422 Unprocessable Entity — boolean není platná hodnota pro integer pole; code = VALIDATION_ERROR.
Skutečný výsledek	API vrátilo 201 Created. Student byl vytvořen a hodnota true byla v databázi automaticky konvertována na 1.
Stav	✗ Failed

POST TC_POST_27 — Vytvoření studenta s numberOfFailedStudies = "abc" (string)	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{ "firstName": "Voldemort", "lastName": "Riddle", "email": "v.riddle@seznam.cz", "isEuCitizen": true, "numberOfFailedStudies": "abc" }
DBeaver příkaz	<code>SELECT * FROM students WHERE email = 'v.riddle@seznam.cz'</code>
Očekávaný výsledek	422 Unprocessable Entity — string není platná hodnota pro integer pole; code = VALIDATION_ERROR.
Skutečný výsledek	API vrátilo 422 Unprocessable Entity a záznam v DB nevznikl.
Stav	✓ Passed

POST TC_POST_28 — Vytvoření studenta s isEuCitizen = "true" (string místo boolean)	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{ "firstName": "Minerva", "lastName": "McGonagallová", "email": "minerva.mc@seznam.cz", "isEuCitizen": "true", "numberOfFailedStudies": 0 }
DBeaver příkaz	<code>SELECT * FROM students WHERE email = 'minerva.mc@seznam.cz'</code>
Očekávaný výsledek	422 Unprocessable Entity — isEuCitizen musí být boolean; code = VALIDATION_ERROR.
Skutečný výsledek	API vrátilo 201 Created. Student byl vytvořen a hodnota true byla v systému akceptována jako platný boolean.
Stav	✗ Failed

POST TC_POST_29 — Vytvoření studenta s isEuCitizen = 1 (integer místo boolean)	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ.

	- Potvrdím tlačítkem SEND. - Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{ "firstName": "Rubeus", "lastName": "Hagrid", "email": "rubeus.h@seznam.cz", "isEuCitizen": 1, "numberOfFailedStudies": 0 }
DBeaver příkaz	<code>SELECT * FROM students WHERE email = 'rubeus.h@seznam.cz'</code>
Očekávaný výsledek	422 Unprocessable Entity — integer není platná hodnota pro boolean pole; code = VALIDATION_ERROR.
Skutečný výsledek	API vrátilo 201 Created. Student byl vytvořen a hodnota 1 byla automaticky konvertována na true.
Stav	✗ Failed

POST TC_POST_30 — Vytvoření studenta s isEuCitizen = "yes" (neplatný string)	
Kroky	- Vstupní podmínky jsou zadány. - Pole BODY obsahuje údaje pro tento testovací případ. - Potvrdím tlačítkem SEND. - Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{ "firstName": "Alastor", "lastName": "Moody", "email": "alastor.m@seznam.cz", "isEuCitizen": "yes", "numberOfFailedStudies": 0 }
DBeaver příkaz	<code>SELECT * FROM students WHERE email = 'alastor.m@seznam.cz'</code>
Očekávaný výsledek	422 Unprocessable Entity — "yes" není boolean; code = VALIDATION_ERROR.
Skutečný výsledek	API vrátilo 201 Created, student byl vytvořen a hodnota "yes" byla v systému automaticky konvertována na true.
Stav	✗ Failed

POST TC_POST_31 — Vytvoření studenta s isEuCitizen = "" (prázdný string)	
Kroky	- Vstupní podmínky jsou zadány. - Pole BODY obsahuje údaje pro tento testovací případ. - Potvrdím tlačítkem SEND. - Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{ "firstName": "Nymphadora", "lastName": "Tonks", "email": "tonks.n@seznam.cz", "isEuCitizen": "", "numberOfFailedStudies": 0 }
DBeaver příkaz	<code>SELECT * FROM students WHERE email = 'tonks.n@seznam.cz'</code>
Očekávaný výsledek	422 Unprocessable Entity — prázdný string není boolean; code = VALIDATION_ERROR.
Skutečný výsledek	API vrátilo 422 Unprocessable Entity, záznam v DB nevznikl.
Stav	✓ Passed

POST TC_POST_32 — Vytvoření studenta s duplicitním e-mailem	
Kroky	- Vstupní podmínky jsou zadány. - Pole BODY obsahuje údaje pro tento testovací případ. - Potvrdím tlačítkem SEND. - Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{ "firstName": "Neville", "lastName": "Longbottom", "email": "neville.l@seznam.cz", "isEuCitizen": true, "numberOfFailedStudies": 0 }

DBeaver příkaz	<code>SELECT COUNT(*) FROM students WHERE email = 'kristina.b@seznam.cz'</code>
Očekávaný výsledek	400 Bad Request — email již existuje; duplicitní záznam nesmí vzniknout.
Skutečný výsledek	API vrátilo 400 Bad Request, záznam v DB nevznikl.
Stav	✓ Passed

Hraniční testy (TC_POST_33 – TC_POST_46)

POST TC_POST_33 — Vytvoření studenta s isEuCitizen = null	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{ "firstName": "Albus", "lastName": "Dumbledore", "email": "albus.d@seznam.cz", "isEuCitizen": null, "numberOfFailedStudies": 0 }
DBeaver příkaz	<code>SELECT * FROM students WHERE email = 'albus.d@seznam.cz'</code>
Očekávaný výsledek	201 Created — null je povolená hodnota; musí být zachována v response i DB (nesmí být konvertována na false).
Skutečný výsledek	API vrátilo 201 Created.
Stav	✓ Passed

POST TC_POST_34 — Vytvoření studenta s numberOfFailedStudies = null	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{ "firstName": "Dobby", "lastName": "Elfo", "email": "dobby.e@seznam.cz", "isEuCitizen": true, "numberOfFailedStudies": null }
DBeaver příkaz	<code>SELECT * FROM students WHERE email = 'dobby.e@seznam.cz'</code>
Očekávaný výsledek	201 Created — null je povolená hodnota; musí být zachována v DB (nesmí být konvertována na 0).
Skutečný výsledek	API vrátilo 201 Created.
Stav	✓ Passed

POST TC_POST_35 — Vytvoření studenta s extrémně vysokou hodnotou numberOfFailedStudies	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.

Body	{ "firstName": "Tom", "lastName": "Riddle", "email": "tom.riddle@seznam.cz", "isEuCitizen": true, "numberOfFailedStudies": 1478523698741 }
DBeaver příkaz	<code>SELECT * FROM students WHERE email = 'tom.riddle@seznam.cz'</code>
Očekávaný výsledek	201 Created (pokud je hodnota akceptována) nebo 422 (pokud je hodnota příliš velká).
Skutečný výsledek	API vrátilo 500 Internal Server Error, server nedokázal zpracovat požadavek s vysokým číslem.
Stav	✗ Failed

POST TC_POST_36 — Vytvoření studenta s e-mailem bez TLD	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{ "firstName": "Cedric", "lastName": "Diggory", "email": "cedric.d@seznam", "isEuCitizen": true, "numberOfFailedStudies": 0 }
DBeaver příkaz	<code>SELECT * FROM students WHERE email = 'cedric.d@seznam'</code>
Očekávaný výsledek	201 Created — email je string bez validace formátu; API záznam přijme. Případně 422.
Skutečný výsledek	API vrátilo 201 Created.
Stav	✓ Passed

POST TC_POST_37 — Vytvoření studenta s e-mailem bez znaku @	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{ "firstName": "Fred", "lastName": "Weasley", "email": "fred.weasley.seznam.cz", "isEuCitizen": true, "numberOfFailedStudies": 0 }
DBeaver příkaz	<code>SELECT * FROM students WHERE email = 'fred.weasley.seznam.cz'</code>
Očekávaný výsledek	201 Created — email je string bez validace. Případně 422.
Skutečný výsledek	API vrátilo 201 Created.
Stav	✓ Passed

POST TC_POST_38 — Vytvoření studenta s e-mailem pouze s doménovou částí (@seznam.cz)	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{ "firstName": "George", "lastName": "Weasley", "email": "@seznam.cz", "isEuCitizen": true, "numberOfFailedStudies": 0 }
DBeaver příkaz	<code>SELECT * FROM students WHERE email = '@seznam.cz'</code>

Očekávaný výsledek	201 Created — email je string bez validace. Případně 422.
Skutečný výsledek	API vrátilo 201 Created.
Stav	✓ Passed

POST TC_POST_39 — Vytvoření studenta s e-mailem s dvojitým @@	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{ "firstName": "Percy", "lastName": "Weasley", "email": "percy@@seznam.cz", "isEuCitizen": true, "numberOfFailedStudies": 0 }
DBeaver příkaz	<code>SELECT * FROM students WHERE email = 'percy@@seznam.cz'</code>
Očekávaný výsledek	201 Created — email je string bez validace. Případně 422.
Skutečný výsledek	API vrátilo 201 Created.
Stav	✓ Passed

POST TC_POST_40 — Vytvoření studenta s e-mailem obsahujícím mezeru	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{ "firstName": "Arthur", "lastName": "Weasley", "email": "arthur @seznam.cz", "isEuCitizen": true, "numberOfFailedStudies": 0 }
DBeaver příkaz	<code>SELECT * FROM students WHERE email = 'arthur @seznam.cz'</code>
Očekávaný výsledek	201 Created — email je string bez validace. Případně 422.
Skutečný výsledek	API vrátilo 201 Created.
Stav	✓ Passed

POST TC_POST_41 — Vytvoření studenta s prázdným tělem requestu	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{}
DBeaver příkaz	<code>SELECT COUNT(*) FROM students</code>
Očekávaný výsledek	201 Created — žádné pole není povinné; všechna pole jsou null nebo mají výchozí hodnoty.
Skutečný výsledek	API vrátilo 201 Created.

Stav	✓ Passed
------	----------

POST TC_POST_42 — Vytvoření studenta se záměrně poškozeným JSON	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	"firstName": "Bellatrix", "lastName": "Lestrangeová" "email": "bella.l@seznam.cz"
DBeaver příkaz	<code>SELECT * FROM students WHERE email = 'bella.l@seznam.cz'</code>
Očekávaný výsledek	422 Unprocessable Entity — poškozený JSON; code = VALIDATION_ERROR. Záznam v DB nevznikne.
Skutečný výsledek	API vrátilo 422 Unprocessable Entity, záznam v DB nebyl vytvořen.
Stav	✓ Passed

POST TC_POST_43 — Vytvoření studenta s neočekávaným polem v requestu	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{ "firstName": "Cedric", "lastName": "Diggory", "email": "cedric.d2@seznam.cz", "isEuCitizen": true, "numberOfFailedStudies": 0, "unknownField": "testovací hodnota" }
DBeaver příkaz	<code>SELECT * FROM students WHERE email = 'cedric.d2@seznam.cz'</code>
Očekávaný výsledek	422 Unprocessable Entity — neočekávané pole; code = VALIDATION_ERROR.
Skutečný výsledek	API vrátilo 201 Created. Student byl vytvořen, ale pole unknownField bylo ignorováno a v DB se neuložilo.
Stav	✗ Failed

POST TC_POST_44 — Vytvoření studenta s firstName = null (explicitní null)	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{ "firstName": null, "lastName": "Granger", "email": "herm.g@seznam.cz", "isEuCitizen": true, "numberOfFailedStudies": 0 }
DBeaver příkaz	<code>SELECT * FROM students WHERE email = 'herm.g@seznam.cz'</code>
Očekávaný výsledek	201 Created — null je povolená hodnota pro firstName; musí být zachována v DB.
Skutečný výsledek	API vrátilo 201 Created.
Stav	✓ Passed

POST TC_POST_45 — Vytvoření studenta s email = null (explicitní null)	
---	--

Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{ "firstName": "Remus", "lastName": "Lupin", "email": null, "isEuCitizen": true, "numberOfFailedStudies": 0 }
DBeaver příkaz	<code>SELECT * FROM students WHERE first_name = 'Remus' AND last_name = 'Lupin'</code>
Očekávaný výsledek	201 Created — null je povolená hodnota pro email; musí být zachována v DB.
Skutečný výsledek	API vrátilo 201 Created.
Stav	✓ Passed

POST TC_POST_46 — Vytvoření duplicitního studenta bez e-mailu	
Kroky	1. Vstupní podmínky jsou zadány. 2. Pole BODY obsahuje údaje pro tento testovací případ. 3. Potvrdím tlačítkem SEND. 4. Výsledek ověřím v DBeaver pomocí SQL příkazu.
Body	{ "firstName": "Monica", "lastName": "Gellerová", "isEuCitizen": true, "numberOfFailedStudies": 0 }
DBeaver příkaz	<code>SELECT * FROM students WHERE first_name = 'Monica' AND last_name = 'Gellerová'</code>
Očekávaný výsledek	201 Created — dle dokumentace není žádné pole povinné. Student je vytvořen s email = null.
Skutečný výsledek	API vrátilo 201 Created s email = null.
Stav	✓ Passed

► Exekuce testů

Přehled všech provedených testů s výsledky. Datum, skutečný status a stav po testování jsou doplněny dle výsledků z provedených testů.

Testovací scénáře byly provedeny. Celkem testovacích případů: 96.

Legenda	Popis
PASS	Test proběhl dle očekávání.
FAIL	Skutečný výsledek se liší od očekávaného.

GET testy

RUN ID	Datum	TC ID	Oček. status	Výsl. status	Stav	Poznámka
RUN-GET_01	07.06.2026	TC_GET_01	200 OK	200 OK	PASS	
RUN-GET_02	07.06.2026	TC_GET_02	200 OK	200 OK	PASS	
RUN-GET_03	07.06.2026	TC_GET_03	200 OK	200 OK	PASS	
RUN-GET_04	07.06.2026	TC_GET_04	404 / 422	500 Internal Server Error	FAIL	Viz BUG_GET_01
RUN-GET_05	07.06.2026	TC_GET_05	404 / 422	500 Internal Server Error	FAIL	Viz BUG_GET_01
RUN-GET_06	07.06.2026	TC_GET_06	404 / 422	500 Internal Server Error	FAIL	Viz BUG_GET_02
RUN-GET_07	07.06.2026	TC_GET_07	422	422 Unprocessable Entity	PASS	
RUN-GET_08	07.06.2026	TC_GET_08	422	422 Unprocessable Entity	PASS	
RUN-GET_09	07.06.2026	TC_GET_09	422	422 Unprocessable Entity	PASS	
RUN-GET_10	07.06.2026	TC_GET_10	422	405 Method Not Allowed	FAIL	Neočekávaný stavový kód
RUN-GET_11	07.06.2026	TC_GET_11	404 / 422	500 Internal Server Error	FAIL	Viz BUG_GET_01
RUN-GET_12	07.06.2026	TC_GET_12	404	200 OK	FAIL	Viz BUG_DEL_01

DELETE testy

RUN ID	Datum	TC ID	Oček. status	Výsl. status	Stav	Poznámka
RUN-DEL_01	07.06.2026	TC_DEL_01	200 OK	200 OK	PASS	
RUN-DEL_02	07.06.2026	TC_DEL_02	200 OK → 404	200 OK → 200 OK	FAIL	Viz BUG_DEL_01
RUN-DEL_03	07.06.2026	TC_DEL_03	200 OK	200 OK	PASS	

RUN-DEL_04	07.06.2026	TC_DEL_04	404 / 422	500 Internal Server Error	FAIL	Viz BUG_DEL_02
RUN-DEL_05	07.06.2026	TC_DEL_05	404 / 422	500 Internal Server Error	FAIL	Viz BUG_DEL_02
RUN-DEL_06	07.06.2026	TC_DEL_06	422	500 Internal Server Error	FAIL	Viz BUG_DEL_04
RUN-DEL_07	07.06.2026	TC_DEL_07	422	422 Unprocessable Entity	PASS	
RUN-DEL_08	07.06.2026	TC_DEL_08	422	422 Unprocessable Entity	PASS	
RUN-DEL_09	07.06.2026	TC_DEL_09	422	422 Unprocessable Entity	PASS	
RUN-DEL_10	07.06.2026	TC_DEL_10	200 OK → 404	200 OK → 200 OK	FAIL	Viz BUG_DEL_03
RUN-DEL_11	07.06.2026	TC_DEL_11	200 OK → 200 OK	Poškozena data B	FAIL	Viz BUG_DEL_05 (nový)
RUN-DEL_12	07.06.2026	TC_DEL_12	404	500 Internal Server Error	FAIL	Viz BUG_DEL_02

PUT testy

RUN ID	Datum	TC ID	Oček. status	Výsl. status	Stav	Poznámka
RUN-PUT_01	07.06.2026	TC_PUT_01	200 OK	200 OK	PASS	
RUN-PUT_02	07.06.2026	TC_PUT_02	200 OK	200 OK	PASS	
RUN-PUT_03	07.06.2026	TC_PUT_03	200 OK	200 OK	PASS	
RUN-PUT_04	07.06.2026	TC_PUT_04	200 OK	200 OK	PASS	
RUN-PUT_05	07.06.2026	TC_PUT_05	200 OK	200 OK	PASS	
RUN-PUT_06	07.06.2026	TC_PUT_06	200 OK	200 OK	PASS	
RUN-PUT_07	07.06.2026	TC_PUT_07	200 OK	200 OK (zipCode nepřepsáno)	PASS	Viz poznámka TC
RUN-PUT_08	07.06.2026	TC_PUT_08	200 OK	422 Unprocessable Entity	FAIL	Viz BUG_PUT_02
RUN-PUT_09	07.06.2026	TC_PUT_09	200 OK	422 Unprocessable Entity	FAIL	Viz BUG_PUT_02
RUN-PUT_10	07.06.2026	TC_PUT_10	200 OK	422 Unprocessable Entity	FAIL	Viz BUG_PUT_02
RUN-PUT_11	07.06.2026	TC_PUT_11	200 OK	422 Unprocessable Entity	FAIL	Viz BUG_PUT_02
RUN-PUT_12	07.06.2026	TC_PUT_12	200 OK	200 OK	PASS	
RUN-PUT_13	07.06.2026	TC_PUT_13	404	404 Not Found	PASS	

RUN-PUT_14	07.06.2026	TC_PUT_14	404	404 Not Found	PASS	
RUN-PUT_15	07.06.2026	TC_PUT_15	422	404 Not Found	FAIL	Viz BUG_PUT_05
RUN-PUT_16	07.06.2026	TC_PUT_16	422	422 Unprocessable Entity	PASS	
RUN-PUT_17	07.06.2026	TC_PUT_17	422	422 Unprocessable Entity	PASS	
RUN-PUT_18	07.06.2026	TC_PUT_18	422	422 Unprocessable Entity	PASS	
RUN-PUT_19	07.06.2026	TC_PUT_19	422	422 Unprocessable Entity	PASS	
RUN-PUT_20	07.06.2026	TC_PUT_20	422	422 Unprocessable Entity	PASS	
RUN-PUT_21	07.06.2026	TC_PUT_21	422	200 OK (string ignorován)	FAIL	Viz BUG_PUT_04
RUN-PUT_22	07.06.2026	TC_PUT_22	422	422 Unprocessable Entity	PASS	
RUN-PUT_23	07.06.2026	TC_PUT_23	422	422 Unprocessable Entity	PASS	
RUN-PUT_24	07.06.2026	TC_PUT_24	422	422 Unprocessable Entity	PASS	
RUN-PUT_25	07.06.2026	TC_PUT_25	400	200 OK	FAIL	Viz BUG_PUT_03
RUN-PUT_26	07.06.2026	TC_PUT_26	200 OK	200 OK	PASS	

POST testy

RUN ID	Datum	TC ID	Oček. status	Výsl. status	Stav	Poznámka
RUN-POST_01	07.06.2026	TC_POST_01	201 Created	200 OK	FAIL	Viz BUG_POST_01
RUN-POST_02	07.06.2026	TC_POST_02	201 Created	201 Created	PASS	
RUN-POST_03	07.06.2026	TC_POST_03	201 Created	201 Created	PASS	
RUN-POST_04	07.06.2026	TC_POST_04	201 Created	201 Created	PASS	
RUN-POST_05	07.06.2026	TC_POST_05	201 Created	201 Created	PASS	
RUN-POST_06	07.06.2026	TC_POST_06	201 Created	201 Created	PASS	
RUN-POST_07	07.06.2026	TC_POST_07	201 Created	201 Created	PASS	
RUN-POST_08	07.06.2026	TC_POST_08	201 Created	201 Created	PASS	

RUN-POST_09	07.06.2026	TC_POS T_09	201 Created	201 Created	PASS	
RUN-POST_10	07.06.2026	TC_POS T_10	201 Created	201 Created	PASS	
RUN-POST_11	07.06.2026	TC_POS T_11	201 Created	201 Created	PASS	
RUN-POST_12	07.06.2026	TC_POS T_12	201 Created	201 Created	PASS	
RUN-POST_13	07.06.2026	TC_POS T_13	201 Created	201 Created	PASS	
RUN-POST_14	07.06.2026	TC_POS T_14	201 Created	201 Created	PASS	
RUN-POST_15	07.06.2026	TC_POS T_15	201 Created	201 Created	PASS	
RUN-POST_16	07.06.2026	TC_POS T_16	201 Created	201 Created	PASS	
RUN-POST_17	07.06.2026	TC_POS T_17	201 Created	201 Created	PASS	
RUN-POST_18	07.06.2026	TC_POS T_18	201 Created	201 Created	PASS	
RUN-POST_19	07.06.2026	TC_POS T_19	201 Created	201 Created	PASS	
RUN-POST_20	07.06.2026	TC_POS T_20	201 Created	201 Created	PASS	
RUN-POST_21	07.06.2026	TC_POS T_21	201 Created	201 Created	PASS	
RUN-POST_22	07.06.2026	TC_POS T_22	201 Created	201 Created	PASS	
RUN-POST_23	07.06.2026	TC_POS T_23	201 Created	201 Created	PASS	
RUN-POST_24	07.06.2026	TC_POS T_24	422	422 Unprocessable Entity	PASS	
RUN-POST_25	07.06.2026	TC_POS T_25	422	422 Unprocessable Entity	PASS	
RUN-POST_26	07.06.2026	TC_POS T_26	422	201 Created	FAIL	Viz BUG_POST_05
RUN-POST_27	07.06.2026	TC_POS T_27	422	422 Unprocessable Entity	PASS	
RUN-POST_28	07.06.2026	TC_POS T_28	422	201 Created	FAIL	Viz BUG_POST_03
RUN-POST_29	07.06.2026	TC_POS T_29	422	201 Created	FAIL	Viz BUG_POST_03
RUN-POST_30	07.06.2026	TC_POS T_30	422	201 Created	FAIL	Viz BUG_POST_03
RUN-POST_31	07.06.2026	TC_POS T_31	422	422 Unprocessable Entity	PASS	
RUN-POST_32	07.06.2026	TC_POS T_32	400	400 Bad Request	PASS	
RUN-POST_33	07.06.2026	TC_POS T_33	201 Created	201 Created	PASS	
RUN-POST_34	07.06.2026	TC_POS T_34	201 Created	201 Created	PASS	

RUN-POST_35	07.06.2026	TC_POS T_35	201 / 422	500 Internal Server Error	FAIL	Viz BUG_POST_07
RUN-POST_36	07.06.2026	TC_POS T_36	201 Created	201 Created	PASS	
RUN-POST_37	07.06.2026	TC_POS T_37	201 Created	201 Created	PASS	
RUN-POST_38	07.06.2026	TC_POS T_38	201 Created	201 Created	PASS	
RUN-POST_39	07.06.2026	TC_POS T_39	201 Created	201 Created	PASS	
RUN-POST_40	07.06.2026	TC_POS T_40	201 Created	201 Created	PASS	
RUN-POST_41	07.06.2026	TC_POS T_41	201 Created	201 Created	PASS	
RUN-POST_42	07.06.2026	TC_POS T_42	422	422 Unprocessable Entity	PASS	
RUN-POST_43	07.06.2026	TC_POS T_43	422	201 Created (pole ignorováno)	FAIL	Viz BUG_POST_08
RUN-POST_44	07.06.2026	TC_POS T_44	201 Created	201 Created	PASS	
RUN-POST_45	07.06.2026	TC_POS T_45	201 Created	201 Created	PASS	
RUN-POST_46	07.06.2026	TC_POS T_46	201 Created	201 Created	PASS	

Bug report

Na základě provedených testů bylo nalezeno 21 chyb. Níže jsou uvedeny jejich popis, závažnost a odkaz na příslušné testovací případy.

Bug ID	Popis chyby	Prav děp.	Záva žnos t	TC ID
BUG_POS T_01	POST /students neukládá pole zipCode. Při vytvoření studenta s hodnotou zipCode = "43019" se pole nepropíše do response ani do DB — v DB je uložena prázdná hodnota.	HIGH	HIGH	TC_POST_01
BUG_POS T_02	POST /students nevaliduje formát pole email. API akceptuje hodnoty bez @ (fred.weasley.seznam.cz), s dvojitým @@ (percy@@seznam.cz), s mezerou (arthur @seznam.cz) a s neplatnou doménou. Pole email je v kontraktu definováno jako string null bez validačních pravidel. Doporučení: doplnit kontrakt o format: email.	MEDI UM	MEDI UM	TC_POST_36–40
BUG_POS T_03	POST /students neprovádí striktní typovou validaci pole isEuCitizen. Stringové hodnoty "true" a "yes" i integer 1 jsou automaticky konvertovány na boolean true místo vrácení validační chyby 422.	HIGH	HIGH	TC_POST_28, TC_POST_29, TC_POST_30
BUG_POS T_04	POST /students při zaslání isEuCitizen = null akceptuje požadavek a vrací 201 Created. Při DB verifikaci je nutné ověřit, zda hodnota null je skutečně zachována a není automaticky konvertována na false. Doporučení: doplnit explicitní assertion na hodnotu v DB response.	HIGH	MEDI UM	TC_POST_33
BUG_POS T_05	POST /students nevaliduje neplatné hodnoty pole numberOfFailedStudies. Záporné (-5), desetinné (5.5), textové ("abc") i boolean (true) hodnoty jsou přijaty a uloženy jako 0 místo vrácení chyby 422.	HIGH	HIGH	TC_POST_24–27
BUG_POS T_06	POST /students při zaslání numberOfFailedStudies = null akceptuje požadavek a vrací 201 Created. Při DB verifikaci je nutné ověřit, zda hodnota null je skutečně zachována a není automaticky konvertována na 0. Doporučení: doplnit explicitní assertion na hodnotu v DB response.	HIGH	MEDI UM	TC_POST_34
BUG_POS T_07	POST /students při extrémně vysoké hodnotě numberOfFailedStudies (1478523698741) vrací 500 Internal Server Error namísto 201 Created nebo 422. Server nedokáže zpracovat hodnotu přesahující rozsah integer.	HIGH	HIGH	TC_POST_35
BUG_POS T_08	POST /students akceptuje request s neočekávaným polem (unknownField) a tiše jej ignoruje namísto vrácení validační chyby 422.	MEDI UM	MEDI UM	TC_POST_43
BUG_POS T_09	POST /students při zaslání lastName = "" (prázdný string) ukládá hodnotu jako null místo prázdného stringu. Stejné chování bylo identifikováno u pole email při prázdném stringu.	HIGH	MEDI UM	TC_POST_08
BUG_GET _01	GET /students/{id} vrací 500 Internal Server Error místo 404 Not Found pro neexistující nebo hraniční ID (id = 0, neexistující ID). Response obsahuje NOT_FOUND v těle, ale HTTP status kód je 500.	HIGH	HIGH	TC_GET_04, TC_GET_05
BUG_GET _02	GET /students/{id} vrací 500 Internal Server Error místo 404 nebo 422 pro záporné ID. Záporná	HIGH	HIGH	TC_GET_06

	hodnota není validačně odmítnuta, ale zpracována jako dotaz na neexistující záznam.			
BUG_DEL_01	DELETE /students/{id} vrací status 200 OK, ale záznam v DB není skutečně smazán. Response potvrzuje smazání, ale SELECT po DELETE stále vrací původní záznam.	HIGH	HIGH	TC_DEL_01
BUG_DEL_02	DELETE /students/{id} vrací 500 Internal Server Error místo 404 Not Found pro neexistující ID (id = 1000) nebo hraniční ID (id = 0). Response obsahuje NOT_FOUND v těle.	HIGH	HIGH	TC_DEL_04, TC_DEL_05
BUG_DEL_03	DELETE /students/{id} vrací 200 OK při opakovaném smazání stejného ID místo 404 Not Found. Pravděpodobně souvisí s BUG_DEL_01 — záznam nebyl při prvním volání skutečně smazán.	HIGH	HIGH	TC_DEL_10
BUG_DEL_04	DELETE /students/{id} vrací 500 Internal Server Error místo 422 pro záporné ID. Záporná hodnota není validačně odmítnuta.	HIGH	HIGH	TC_DEL_06
BUG_DEL_05	DELETE /students/{id} způsobuje poškození dat sousedního záznamu. Po smazání studenta A (ID 173) byly v záznamu studenta B (ID 174) nastaveny hodnoty jména a příjmení na null. Kritická chyba integrity dat.	HIGH	HIGH	TC_DEL_11
BUG_PUT_01	PUT /students/{id} při částečné aktualizaci (pouze jedno pole) nekonzistentně ukládá pole isEuCitizen — hodnota se v některých případech změní na false i přesto, že nebyla součástí requestu. Chování je intermitentní a reprodukovatelné pouze při opakované exekuci. Doporučení: ověřit serverovou logiku partial update.	HIGH	HIGH	TC_PUT_02, TC_PUT_26
BUG_PUT_02	PUT /students/{id} odmítá platné null hodnoty pro pole lastName, email, isEuCitizen a numberOfFailedStudies s obecným chybovým hlášením místo jejich akceptování dle kontraktu.	HIGH	MEDIUM	TC_PUT_08-11
BUG_PUT_03	PUT /students/{id} akceptuje neplatný email (nevalidní formát) a duplicitní email jiného studenta bez vrácení chyby. V DB mohou existovat dva studenti se stejným emailem.	HIGH	HIGH	TC_PUT_04, TC_PUT_25
BUG_PUT_04	PUT /students/{id} akceptuje string "true" pro pole isEuCitizen místo vyžadování boolean hodnoty.	HIGH	MEDIUM	TC_PUT_21
BUG_PUT_05	PUT /students/{id} vrací 404 Not Found místo 422 Unprocessable Entity pro záporné ID. Záporná hodnota není validačně odmítnuta — stejné chování jako BUG_GET_02 a BUG_DEL_04.	HIGH	HIGH	TC_PUT_15

— Konec dokumentu —